

Задача 3. ООП, декораторы, файлы

Задача должна быть выполнена на языке Python, используя пройденный материал. Условие задачи и автор должны быть указаны в комментариях. Также должны быть «обычные» комментарии в коде.

Создать консольное приложение с меню, которое будет демонстрировать все действия, которые можно выполнить над указанным классом. Задача не должна завершаться после одного действия — вместо этого каждый раз должна возвращаться в меню до выбора пункта «завершить».

Пользователь (преподаватель) должен понимать что ему нужно делать из консоли, а не из условия задачи, кода программы или ваших устных подсказок.

Во всех задачах должна быть предусмотрена возможность ввода и вывода из файла и с консоли. Задачи должны быть разбиты на функции.

Классы вынести в отдельный модуль. Поля сделать закрытыми и с геттерами; сеттеры дописать по мере необходимости. У класса должны быть определены инициализатор и метод приведения к строке.

Декорирование происходит через @.

Условия:

1. Написать декоратор, сохраняющий название функции (с помощью `__name__`) в список. Список поступает как параметр декоратора.

Написать класс «Item» с полями «название», «описание», «цена», «число зарядов». Перегрузить операторы «+=» и «-=», изменяющие число зарядов на заданное целочисленное значение.

Программа должна содержать список айтемов, которые можно добавлять, удалять, изменять число зарядов и печатать. Декоратор применить ко всем методам класса.

2. Написать декоратор_1, который приводит строку-результат функции к верхнему регистру. Написать декоратор_2, который добавляет кавычки-елочки вокруг строки-результата.

Написать класс «Truck» с полями «марка», «модель», «мощность двигателя», «объем бака», «текущий уровень топлива». Перегрузить операторы «+=» и «-=», изменяющие текущий уровень топлива на заданное вещественное значение. Перегрузить операторы сравнения, сравнивающие два тягача по мощности двигателя.

Программа должна содержать список тягачей, которые можно добавлять, удалять, изменять уровень топлива и печатать. Декораторы 1 и 2 применить ко геттеру марки. Декоратор 2 применить к геттеру модели.

3. Написать декоратор с двумя параметрами: значение для True, значение для False. Он заменяет логическое значение-результат функции на значение одного из параметров.

Написать класс «Пациент» с полями «ФИО», «пол» (bool), «диагноз», «жив ли» (bool).

Программа должна содержать список пациентов, которых можно добавлять, удалять и печатать. Декоратор применить к геттерам «пол» и «жив ли».

4. Написать декоратор, который автоматически подставляет значение во второй параметр функции, тем самым уменьшая число ее аргументов. Декоратор подставляет строку-цвет: один из семи цветов радуги — при очередном вызове цвет автоматически выбирается «следующим» (т.е. к-о-ж-з-г-с-ф-к...). Для этого декоратор должен «помнить» предыдущее значение. Декоратор вынести в отдельный модуль.

Написать класс «Рыцарь» с полями «имя», «цвет знамени» и «титул». Добавить сеттер цвета знамени.

Программа должна содержать список рыцарей, которых можно добавлять, удалять и печатать. Декоратор применить к инициализатору рыцаря и сеттеру знамени.

5. Написать декоратор «удача» с параметрами N и M: с шансом N% увеличивает результат в M раз.

Написать класс «Гладиатор» с полями «имя», «здоровье» и «урон». Декорировать геттер урона.

Задача должна мочь настраивать двух гладиаторов и устраивать поединок между ними.

6. Написать декоратор с параметром-коэффициентом. Он переводит результат из одной величины в другую.

Написать класс «Округ» с полями «название» и «площадь». Переопределить операторы «+=», «-=», «*=» и «/=», изменяющие его площадь на вещественное число.

Программа должна содержать список округов, которые можно добавлять, удалять, изменять площадь и печатать. Декоратором, который переводит м² в попугай² (один попуг = 19,8см). декорировать геттер площади.

7. Написать декоратор, который помимо основного действия печатает «Вы осмелились потревожить *.name от первого аргумента*».

Написать класс «Бог» с полями «имя», «стихия». Добавить метод «выслушать молитву», который принимает строку и ничего с ней не делает. Переопределить операторы «==» и «!=», сравнивающие богов по имени.

Написать класс «Пантеон богов», который содержит список богов. Переопределить методы «+= Бог» для добавления и «-= Бог» для удаления, если такой бог найден.

Программа должна содержать пантеон богов, добавлять, удалять богов, печатать пантеон. Декоратор применяется ко всем методам класса «Бог» кроме геттеров и инициализатора.

8. Написать декоратор, который число заменяет на приближенное значение, превращая в строку вида «число единица_измерения (миллионы, тысячи, сотни или десятки)» (то есть, например, 273407 на «273 тысячи», 215 на «2 сотни»).

Написать класс «Стихийное бедствие» с полями «тип» (например, землетрясение или тайфун), «место», «число погибших». Декорировать геттер на «число погибших».

Написать дочерний класс «Техногенная катастрофа», который имеет дополнительное поле «виновник».

Программа должна содержать список бедствий и катастроф, которые можно добавлять, удалять и печатать.

9. Написать декоратор, который в строке-результате функции переставляет первое слово в конец.

Написать класс «Джедай» с полями «имя», «цвет меча», «навык» (enum: Впитывание силы, Лечение, Воздействие на разум, Защита, Похищение силы, Молния, Удушение, Темная ярость), «коронная фраза». Добавить сеттер на навык.

Программа должна содержать список джедаев, которые можно добавлять, удалять, менять навык и печатать. Применить декоратор к геттеру коронной фразы.

10. Написать декоратор с параметром «суффикс», который превращает результат в строку, добавляя суффикс в конец.

Написать класс «Геокоордината» с полями «градусы», «минуты», «секунды». Декорировать все геттеры.

Класс «Метеодатчик» с полями «долгота» и «широта» (геокоординаты), «ID», «температура», «влажность». Декорировать геттеры влажности (%) и температуры (°).

Программа должна содержать список метеодатчиков, которые можно добавлять, удалять и печатать.

11. Написать декоратор, который округляет число до целого.

Написать класс «Аттестат», который хранит ФИО ученика и набор пар «предмет, оценка». Он содержит метод, возвращающий средний балл по предметам (декорировать его). Для добавления оценки за предмет перегрузить оператор «+=». Реализовать магические методы класса-контейнера (len, getitem, setitem, delitem, contains).

Программа должна мочь создавать аттестат и работать с ним.

12. Написать декоратор_1, который функцию от 2 аргументов, превращает в функцию от 1 аргумента, автоматически подставляя вместо первого π из математического модуля.

Написать декоратор_2, который превращает число-результат в строку, состоящую из этого числа с точностью ровно 6 знаков после запятой.

Написать статический класс «Калькулятор площадей» с методами вычисления площадей правильных: 3, 4, 5 и 6-угольников по их стороне; площади круга, которая принимает π и радиус. Декоратор 1 применить к методу нахождения площади круга. Декоратор 2 применить ко всем методам.

Задача должна демонстрировать все возможности класса.

13. Написать декоратор, который переводит год в век (целочисленный).

Написать декоратор, который переводит арабское число (от -40 до $+40$) в римское с подписью «до н.э.» для отрицательных значений.

Написать класс «Историческое событие» с полями «название» (str) и «год» (int). Все поля можно изменять. Применить оба декоратора к геттеру года.

Программа должна содержать список событий, которые можно добавлять, удалять, редактировать и печатать.